```
/* ============================================================
   spinВгувт — логика (Человек 2)
   ============================================================ */

// ---------- Настройки правил игры (легко менять) ----------
const MIN_MARK  = 2;   // минимальная оценка
const MAX_MARK  = 5;   // максимальная оценка
const NEW_AT_MAX = 4;  // выпал твой цвет, а выбрана 5 → добавляется новая оценка
const NEW_AT_MIN = 2;  // выпал твой цвет, а выбрана 2 → добавляется новая оценка
const ZERO_MARK = 5;   // до какой оценки исправляется выбранная при зеро
const LOSE_MARK = 2;   // до какой оценки падает выбранная при проигрыше

// ---------- Состояние ----------
// subjects: [{ id, name, marks: [{ id, v }] }]
let state = { name: "", subjects: [] };
let selectedId = null;   // id выбранной оценки
let bet = null;          // "red" | "black" | "zero"
let spinning = false;
let wheelRotation = 0;
let lastAvg = null;
let freshId = null;      // id только что добавленной оценки (для анимации)
let resetArmed = false;

const STORE = "spinVguvt2";
const $ = id => document.getElementById(id);
const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 7

// ---------- Сохранение ----------
function save() { try { localStorage.setItem(STORE, JSON.stringify(state)); } cat
function load() {
  try {
    const s = JSON.parse(localStorage.getItem(STORE));
    if (s && s.name && Array.isArray(s.subjects) && s.subjects.some(x => x.marks.
      state = s; return true;
    }
  } catch (e) {}
  return false;
}

// ---------- Средний балл ----------
function allMarks() { return state.subjects.flatMap(s => s.marks); }
function mean(marks) {
  return marks.length ? marks.reduce((a, m) => a + m.v, 0) / marks.length : 0;
}
```

```
function updateAverage() {
  const avg = mean(allMarks());
  $("avgValue").textContent = avg.toFixed(2);
  const d = $("avgDelta");
  if (lastAvg !== null && Math.abs(avg - lastAvg) > 0.0001) {
    const diff = avg - lastAvg;
    d.textContent = (diff > 0 ? "▲ +" : "▼ ") + diff.toFixed(2);
    d.className = "delta " + (diff > 0 ? "up" : "down");
  } else {
    d.textContent = ""; d.className = "delta";
  }
  lastAvg = avg;
}

// ---------- Отрисовка предметов и оценок ----------
function escapeHtml(s) {
  return s.replace(/[&<>"']/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"
}

// mode: "login" (можно удалять) или "game" (можно выбирать)
function renderSubjects(container, mode) {
  container.innerHTML = "";
  state.subjects.forEach(sub => {
    const box = document.createElement("div");
    box.className = "subject";

    const head = document.createElement("div");
    head.className = "subj-head";
    head.innerHTML = `<span class="subj-name">${escapeHtml(sub.name)}</span>` +
                     `<span class="subj-avg">ср. ${mean(sub.marks).toFixed(2)}</s
    if (mode === "login") {
      const del = document.createElement("button");
      del.textContent = "Удалить"; del.className = "ghost del";
      del.onclick = () => { state.subjects = state.subjects.filter(x => x.id !==
      head.appendChild(del);
    }
    box.appendChild(head);

    const chips = document.createElement("div");
    chips.className = "chips";
    sub.marks.forEach(m => {
      const c = document.createElement("button");
      c.className = `chip m${m.v}` + (mode === "game" && m.id === selectedId ? "
                    (m.id === freshId ? " fresh" : "");
      c.textContent = m.v;
      if (mode === "game") {
        c.title = "Выбрать для ставки";
```

```
        c.onclick = () => {
          if (spinning) return;
          selectedId = m.id; freshId = null;
          renderSubjects(container, "game"); checkReady();
        };
      } else {
        c.title = "Убрать эту оценку";
        c.onclick = () => {
          sub.marks = sub.marks.filter(x => x.id !== m.id);
          if (!sub.marks.length) state.subjects = state.subjects.filter(x => x.id
          renderLogin();
        };
      }
      chips.appendChild(c);
    });
    box.appendChild(chips);
    container.appendChild(box);
  });
}

// ---------- Вход ----------
function renderLogin() {
  renderSubjects($("loginSubjects"), "login");
  $("startBtn").disabled = !($("nameInput").value.trim() && allMarks().length);
}

// "5 4 4,3 5" или "54435" -> [5,4,4,3,5]
function parseMarks(str) { return (str.match(/[2-5]/g) || []).map(Number); }

function addSubject() {
  const marks = parseMarks($("marksInput").value);
  if (!marks.length) { $("marksInput").focus(); return; }
  const name = $("subjectInput").value.trim() || "Предмет " + (state.subjects.len
  let sub = state.subjects.find(s => s.name.toLowerCase() === name.toLowerCase())
  if (!sub) { sub = { id: uid(), name, marks: [] }; state.subjects.push(sub); }
  marks.forEach(v => sub.marks.push({ id: uid(), v }));
  $("subjectInput").value = ""; $("marksInput").value = "";
  $("subjectInput").focus();
  renderLogin();
}
$("addBtn").onclick = addSubject;
$("marksInput").addEventListener("keydown", e => { if (e.key === "Enter") addSubj
$("subjectInput").addEventListener("keydown", e => { if (e.key === "Enter") $("ma
$("nameInput").oninput = renderLogin;

$("startBtn").onclick = () => {
  state.name = $("nameInput").value.trim();
```

```
  save(); startGame();
};

// Сброс: confirm() в браузерах-песочницах не работает, поэтому подтверждаем втор
$("logoutBtn").onclick = () => {
  const btn = $("logoutBtn");
  if (!resetArmed) {
    resetArmed = true; btn.textContent = "Точно сбросить? Нажми ещё раз";
    setTimeout(() => { resetArmed = false; btn.textContent = "Выйти / сбросить";
    return;
  }
  resetArmed = false; btn.textContent = "Выйти / сбросить";
  try { localStorage.removeItem(STORE); } catch (e) {}
  state = { name: "", subjects: [] };
  selectedId = null; bet = null; lastAvg = null; freshId = null;
  document.querySelectorAll(".bet").forEach(x => x.classList.remove("active"));
  $("result").textContent = "Выбери оценку и цвет";
  $("screenGame").classList.add("hidden");
  $("screenLogin").classList.remove("hidden");
  $("nameInput").value = "";
  renderLogin();
};

function startGame() {
  $("screenLogin").classList.add("hidden");
  $("screenGame").classList.remove("hidden");
  $("hello").textContent = "Привет, " + state.name + "!";
  lastAvg = null;
  renderSubjects($("gameSubjects"), "game");
  updateAverage();
  drawWheel();
}

// ---------- Колесо ----------
// 37 секторов: 0 — зелёный, остальные чередуются красный/чёрный
const N = 37, STEP = 360 / N;
function colorOf(i) { return i === 0 ? "zero" : (i % 2 === 1 ? "red" : "black");

function drawWheel() {
  const c = $("wheel"), ctx = c.getContext("2d");
  const cx = c.width / 2, cy = c.height / 2, r = cx - 6;
  ctx.clearRect(0, 0, c.width, c.height);
  const colors = { red: "#e53935", black: "#212121", zero: "#2e9d4f" };
  for (let i = 0; i < N; i++) {
    const a0 = (-90 + i * STEP) * Math.PI / 180;
    const a1 = (-90 + (i + 1) * STEP) * Math.PI / 180;
    ctx.beginPath(); ctx.moveTo(cx, cy); ctx.arc(cx, cy, r, a0, a1); ctx.closePat
```

```
    ctx.fillStyle = colors[colorOf(i)]; ctx.fill();
    ctx.strokeStyle = "#ffc247"; ctx.lineWidth = 2; ctx.stroke();
    const am = (a0 + a1) / 2;
    ctx.save();
    ctx.translate(cx + Math.cos(am) * r * 0.85, cy + Math.sin(am) * r * 0.85);
    ctx.rotate(am + Math.PI / 2);
    ctx.fillStyle = "#fff"; ctx.font = "bold 20px Arial";
    ctx.textAlign = "center"; ctx.textBaseline = "middle";
    ctx.fillText(i, 0, 0); ctx.restore();
  }
  ctx.beginPath(); ctx.arc(cx, cy, r * 0.25, 0, 7);
  ctx.fillStyle = "#1a1f35"; ctx.fill(); ctx.strokeStyle = "#ffc247"; ctx.stroke(
}

// ---------- Ставки ----------
document.querySelectorAll(".bet").forEach(b => {
  b.onclick = () => {
    if (spinning) return;
    bet = b.dataset.bet;
    document.querySelectorAll(".bet").forEach(x => x.classList.toggle("active", x
    checkReady();
  };
});
function checkReady() { $("spinBtn").disabled = spinning || !(selectedId && bet);

// ---------- Вращение ----------
$("spinBtn").onclick = () => {
  if (spinning || !selectedId || !bet) return;
  spinning = true; checkReady();
  $("result").textContent = "Крутим... 🎡";

  const winIndex = Math.floor(Math.random() * N);
  const jitter = (Math.random() - 0.5) * STEP * 0.7;
  const desired = (360 - (winIndex + 0.5) * STEP + jitter + 360) % 360;
  const cur = ((wheelRotation % 360) + 360) % 360;
  const delta = (desired - cur + 360) % 360;
  wheelRotation += 360 * 5 + delta;
  $("wheel").style.transform = `rotate(${wheelRotation}deg)`;

  setTimeout(() => finishSpin(winIndex), 4100);
};

function findSelected() {
  for (const sub of state.subjects) {
    const mark = sub.marks.find(m => m.id === selectedId);
    if (mark) return { sub, mark };
  }
```

```
  return null;
}

// Главные правила игры
function applyResult(color, found) {
  const { sub, mark } = found;
  const before = mark.v;

  if (color === "zero" && bet === "zero") {
    mark.v = ZERO_MARK;
    return `🎉 Зеро! Оценка ${before} исправлена на ${ZERO_MARK} (${sub.name}).`;
  }
  if (color !== "zero" && bet === color) {
    // Оценка между минимумом и максимумом: просто улучшаем её на 1
    if (before > MIN_MARK && before < MAX_MARK) {
      mark.v = before + 1;
      return `✅ Угадал! Оценка ${before} выросла до ${mark.v} (${sub.name}).`;
    }
    // Оценка максимальная или минимальная: добавляется новая
    const v = before >= MAX_MARK ? NEW_AT_MAX : NEW_AT_MIN;
    const added = { id: uid(), v };
    sub.marks.push(added);
    freshId = added.id;
    return `✅ Угадал! Оценка ${before} крайняя, поэтому в «${sub.name}» добавлен
  }
  mark.v = LOSE_MARK;
  return `❌ Не угадал. Оценка ${before} стала ${LOSE_MARK} (${sub.name}).`;
}

function finishSpin(winIndex) {
  const color = colorOf(winIndex);
  const names = { red: "красное", black: "чёрное", zero: "ЗЕРО" };
  const found = findSelected();
  let msg = `Выпало ${winIndex} — ${names[color]}. `;
  if (found) msg += applyResult(color, found);

  $("result").textContent = msg;
  spinning = false;
  save();
  renderSubjects($("gameSubjects"), "game");
  updateAverage();
  checkReady();
}

// ---------- Старт ----------
if (load()) { startGame(); } else { renderLogin(); }
```